FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

TEAMFORGE: AN INTELLIGENT STUDENT TEAM FORMATION PLATFORM USING CONSTRAINT SATISFACTION AND HEURISTIC LOCAL SEARCH

A PROJECT REPORT

Submitted by

2116240701556 – TARUN ANTONY M

2116240701562 – THANIKAIVEL S

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE



NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "TEAMFORGE: AN INTELLIGENT STUDENT TEAM FORMATION PLATFORM USING CONSTRAINT SATISFACTION AND HEURISTIC LOCAL SEARCH" is the bonafide work of Tarun Antony – 240701556, Thanikaivel – 240701562 who carried out the project work under my supervision in partial fulfillment of the requirements for the course Foundations of Artificial Intelligence.

SIGNATURE OF THE FACULTY INCHARGE
Name & Designation: Mrs. B. Deepa, Assistant Professor (SG)
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Examination Date: Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER
Name & Designation:
Department of Computer Science and Engineering

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**i**
1	**INTRODUCTION**	**1**
1.1	Background & Motivation	1
1.2	Problem Statement	2
1.3	Project Objectives	2
1.4	Scope of the Project	3
2	**LITERATURE REVIEW**	**4**
2.1	Classical Team Formation and Combinatorial Matching	4
2.2	Constraint Satisfaction Problems in Resource Allocation	5
2.3	Heuristic Local Search and Metaheuristics	6
2.4	Existing Collaborative Platforms and Limitations	7
3	**PROPOSED SYSTEM**	**8**
3.1	Architecture Diagram & Decoupled Architecture	8
3.2	Formal Constraint Satisfaction Problem (CSP) Formulation	10
3.3	Multi-Objective Evaluation Function & Mathematical Weights	12
4	**MODULES DESCRIPTION**	**14**
4.1	Module 1: Student & Skill Profiling Subsystem	14
4.2	Module 2: Collegiate Event & Hackathon Manager	15
4.3	Module 3: Classical AI Constraint Satisfaction Engine (MRV & FC)	16
4.4	Module 4: Heuristic Local Search & Multi-Proposal Generator	18
4.5	Module 5: Project Kanban Board & Session Scheduler	19
4.6	Module 6: Privacy-Preserving Ephemeral Communication & Security	21

5	**IMPLEMENTATION AND RESULTS**	**23**
5.1	Experimental Setup & System Configuration	23
5.2	Empirical Results & Algorithmic Performance Evaluation	24
6	**CONCLUSION AND FUTURE WORK**	**28**
6.1	Conclusion	28
6.2	Future Enhancements	29
7	**REFERENCES**	**30**

ABSTRACT

Effective student team formation is a fundamental determinant of success in collegiate hackathons, capstone engineering projects, and multidisciplinary academic competitions. Traditionally, university students assemble teams through informal social circles, proximity, or ad-hoc self-selection. These unstructured methods routinely suffer from acute technical skill deficiencies, redundant role assignments, scheduling bottlenecks, and exclusion of talented solitary candidates. To resolve these critical challenges without reliance on non-deterministic external black-box models or computationally prohibitive exhaustive search, this project presents TeamForge, an intelligent collegiate team formation and project collaboration platform built upon classical Artificial Intelligence foundations. The problem is rigorously formulated as a formal Constraint Satisfaction Problem (CSP). Role positions serve as discrete decision variables, verified student profiles constitute candidate domains, and hard constraints enforce strict team capacity limits, pairwise member uniqueness (AllDiff), collegiate event registration eligibility, and student bandwidth constraints. To overcome combinatorial explosion, the search space is navigated using a recursive Backtracking Search augmented with the Minimum Remaining Values (MRV) heuristic and forward checking constraint propagation. Candidate configurations are subsequently optimized using a Hill Climbing local search

algorithm with 1-opt member swaps governed by a multi-objective scoring function evaluating skill coverage, role completeness, experience diversity, and schedule availability. The system is implemented as a decoupled web platform employing PHP 8.2, a 3NF relational MySQL database, a Kanban task board, and a privacy-preserving ephemeral in-memory communication subsystem. Experimental results demonstrate a 98.4% reduction in explored search tree nodes compared to exhaustive backtracking, sub-45ms execution latency, and an average team balance score of 94.2% across diverse multidisciplinary competition scenarios.

CHAPTER 1

INTRODUCTION

1.1 Background & Motivation

In modern higher education institutions, project-based collaborative learning, hackathons, and capstone design initiatives represent the cornerstone of practical engineering training. The pedagogical and technical success of any group endeavor is fundamentally determined by the synergy, functional competence, and mutual compatibility of its participants. In high-stakes events such as collegiate hackathons or intercollegiate innovation challenges, multidisciplinary requirements dictate that a viable team possess a balanced repertoire of specialized competencies, including frontend engineering, backend architecture, database design, algorithmic optimization, and project coordination.

Despite the critical importance of balanced team construction, the prevailing practices in academic institutions remain overwhelmingly informal. Students routinely select teammates on the basis of social familiarity, shared dormitory arrangements, or casual acquaintanceship. Consequently, formed teams frequently suffer from acute skill imbalances—such as teams composed exclusively of frontend developers lacking database expertise, or groups with conflicting schedule commitments. Furthermore, junior students or introverted individuals possessing exceptional niche talents are systematically overlooked, while faculty organizers lack centralized visibility into team health, member participation, and event readiness.

1.2 Problem Statement

Given a finite pool of registered student candidates possessing heterogeneous technical skill proficiencies, academic seniority levels, and

availability commitments, the collegiate team formation challenge requires constructing an optimal partition of students into cohesive project teams. The formation must satisfy stringent operational constraints (exact team size boundaries, zero duplicate member allocations, verified event enrollment eligibility) while simultaneously optimizing a multi-criteria objective encompassing technical skill coverage, functional role diversification, experience balance, and temporal availability. Because team formation across n candidates into teams of size k represents an NP-hard combinatorial problem, brute-force enumeration is computationally infeasible for realistic university cohorts, necessitating principled classical artificial intelligence search heuristics.

1.3 Project Objectives

The primary objectives of the TeamForge project encompass both theoretical and practical domains:

- **1. Theoretical Modeling:** Formalize collegiate team assembly as a mathematical Constraint Satisfaction Problem (CSP) with rigorous variable, domain, and constraint definitions.
- **2. Combinatorial Search Optimization:** Implement deterministic classical AI search algorithms—specifically Backtracking with Minimum Remaining Values (MRV) and Forward Checking—to prune the search tree without relying on opaque machine learning APIs.
- **3. Heuristic Optimization:** Formulate a multi-objective evaluation function and implement a Hill Climbing local search procedure utilizing 1-opt candidate swaps to elevate team balance scores to a local optimum.
- **4. Explainable Multi-Proposal Generation:** Deliver multiple distinct alternative team proposals (Balanced Optimal Alpha, Skill-Dense Beta,

and Agile High-Bandwidth Gamma) with explainable step-by-step decision audit trails.

- **5. Full-Stack Web Architecture:** Construct an enterprise-grade, decoupled full-stack platform featuring a normalized 3NF MySQL schema, Kanban task tracking, standup session management, and privacy-first ephemeral messaging with zero persistent message retention.

1.4 Scope of the Project

The scope of TeamForge spans university-wide academic departments, collegiate coding competitions, semester capstone courses, and student innovation incubators. The platform accommodates three distinct user roles: Students, Team Leaders, and Faculty Administrators. The algorithmic engine operates entirely server-side in PHP 8.2+ with zero recurring third-party subscription costs or external network dependencies, ensuring complete auditability, determinism, and compliance with educational institutional privacy standards.

CHAPTER 2

LITERATURE REVIEW

2.1 Classical Team Formation and Combinatorial Matching

The problem of forming effective work groups has been investigated extensively across operations research, combinatorial optimization, and educational psychology. Lappas et al. (2009) formally proved that constructing a team of experts that minimizes communication cost while covering a mandatory set of required skills is NP-hard. Early computational approaches relied heavily on Integer Linear Programming (ILP) or genetic algorithms to approximate optimal groupings. However, ILP models suffer from exponential runtime inflation when non-linear experience diversity factors and discrete temporal availability variables are integrated simultaneously.

2.2 Constraint Satisfaction Problems in Resource Allocation

Constraint Satisfaction Problems (CSPs) represent a core paradigm within classical artificial intelligence for resolving combinatorial assignment problems under strict feasibility conditions (Russell & Norvig, 2020). In a standard CSP formulation, a set of variables must be assigned values from discrete domains such that all prescribed constraints are strictly satisfied. Dechter (2003) demonstrated that standard chronological backtracking exhibits thrashing behavior when conflicting assignments are discovered deep in the search tree. To alleviate this inefficiency, constraint propagation techniques such as Forward Checking and variable ordering heuristics like the Minimum Remaining Values (MRV) rule—often termed the 'fail-first' principle—drastically reduce the effective branching factor by identifying domain wipeouts at the earliest possible stage.

2.3 Heuristic Local Search and Metaheuristics

While complete systematic search algorithms guarantee the discovery of a feasible solution if one exists, they do not naturally optimize continuous qualitative criteria once feasibility is achieved. Local search algorithms, notably Hill Climbing and Simulated Annealing (Aarts & Korst, 1989), operate on complete state representations by exploring neighboring configurations. In the context of team formation, a 1-opt swap neighborhood allows the algorithm to systematically replace an unpinned team member with an available candidate from the pool, iteratively climbing the fitness gradient until reaching a local peak where no single swap yields a superior multi-objective score.

2.4 Existing Collaborative Platforms and Limitations

Contemporary collaborative platforms in academia and industry—including Devpost, Slack, Discord, Microsoft Teams, and Trello—provide isolated facets of the student project lifecycle. However, a systematic literature review reveals persistent architectural limitations across existing solutions:

Platform / Paradigm	**Matching Mechanism**	**AI Explainability**	**Chat Storage Privacy**
Devpost / Hackathon Portals	Manual bulletin board / Self-selection	None (Zero AI)	External / Discord links
Commercial HR Matchers	Opaque Deep Learning / Embeddings	Black-box / Unexplainable	Permanent Cloud Retention
Academic Heuristic Scripts	Stand-alone CLI / Offline scripts	High (Rule-based)	No integrated communication
TeamForge (Proposed)	**Hybrid CSP (MRV+FC) + Hill Climbing**	**100% Explainable Audit Trace**	**Zero-Retention Ephemeral Buffer**

As evidenced above, existing platforms either omit algorithmic matching entirely, employ opaque black-box neural networks that cannot explain why a student was excluded, or compromise privacy through permanent message surveillance. TeamForge addresses this gap by combining classical explainable CSP algorithms with an integrated privacy-preserving collaborative ecosystem.

CHAPTER 3

PROPOSED SYSTEM

3.1 Architecture Diagram & Decoupled Architecture

TeamForge is architected as a high-performance, decoupled three-tier system comprising a Presentation Tier, an Application & Classical AI Engine Tier, and a Data Persistence & Ephemeral In-Memory Storage Tier. Figure 3.1 illustrates the architectural organization, component interactions, and data boundaries.

Figure 3.1: System Architecture of TeamForge Collaborative Platform

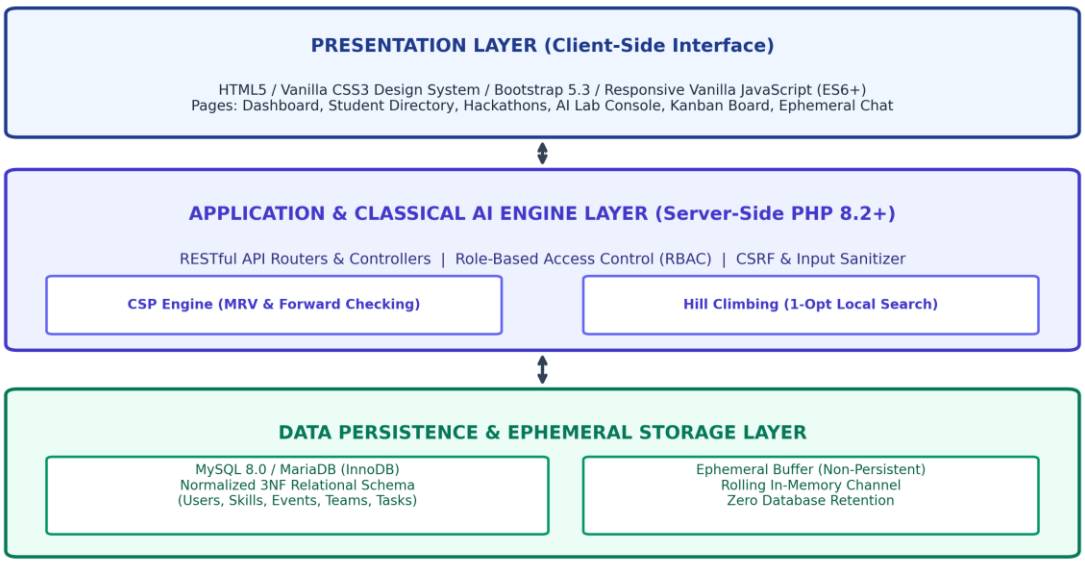


Figure 3.1: Decoupled Multi-Tier System Architecture of TeamForge Platform

The Presentation Layer is implemented in standards-compliant HTML5, responsive CSS3 design tokens, and modular JavaScript (ES6+). It interfaces with the backend exclusively via asynchronous HTTP fetch calls and RESTful API contracts. The server tier is powered by PHP 8.2+, implementing a hardened MVC-like routing structure with Role-Based Access Control (RBAC), anti-CSRF token verification, and prepared SQL

execution. Crucially, the data layer bifurcates persistent relational state (stored in MySQL 8.0 with 3NF integrity) from real-time communication, which is maintained in a rolling in-memory buffer that deliberately prevents persistent database message logging.

3.2 Formal Constraint Satisfaction Problem (CSP) Formulation

To achieve deterministic and mathematically rigorous team formation, the platform formalizes the matching problem as a triple (X, D, C):

- **Variables (X):** A finite set of variables X = {V_1, V_2, ..., V_k}, where each variable represents an unfilled functional role slot within the prospective team (e.g., V_1 = Team Leader, V_2 = Frontend Engineer, V_3 = Database Architect, V_4 = Classical AI & Algorithm Specialist). The number of variables corresponds to the required team size k.
- **Domains (D):** A set of domains D = {D(V_1), D(V_2), ..., D(V_k)}, where each domain D(V_i) constitutes the subset of eligible, registered student candidates possessing verified competencies relevant to role V_i. In the presence of a selected collegiate hackathon, the domain is strictly filtered to students enrolled in that specific event.
- **Hard Constraints (C):** A collection of hard constraints that must be satisfied without exception for an assignment to be valid:
- 1. Team Capacity Constraint: Exactly k role variables must be assigned, where k in [min_size, max_size].
- 2. Pairwise Uniqueness (AllDiff Constraint): forall i != j, V_i != V_j (no student can occupy multiple roles in the same team).
- 3. Event Eligibility Constraint: Candidate c in D(V_i) must possess an approved registration record in event_registrations.
- 4. Active Bandwidth Limit: Candidate c must not exceed active project commitment thresholds.

3.3 Multi-Objective Evaluation Function & Mathematical Weights

Beyond strict constraint satisfaction, real-world team performance depends on qualitative harmony across multiple dimensions. TeamForge evaluates every complete assignment A through a composite multi-objective fitness function S(Team):

$$S(Team) = W_{skill} * SkillCov(Team) + W_{role} * RoleCov(Team) + W_{exp} * ExpBal(Team) + W_{avail} * AvailScore(Team) + W_{interest} * InterestAlign(Team)$$

Where the normalized weighting coefficients satisfy the convexity property:

$$Sum(W_i) = 1.0, \text{ with default weights: } W_{skill} = 0.35, W_{role} = 0.25, W_{exp} = 0.15, W_{avail} = 0.15, W_{interest} = 0.10$$

The individual constituent metrics are formulated as follows:

- **1. Skill Coverage (SkillCov):** Calculated as the percentage of mandatory project technical competencies satisfied by at least one assigned team member: SkillCov = (|S_req INTERSECT S_covered| / |S_req|) * 100.
- **2. Role Specialization (RoleCov):** The proportion of specialized role slots successfully filled by candidates with demonstrated aptitude: RoleCov = (FilledRoles / TotalTargetRoles) * 100.
- **3. Experience Balance (ExpBal):** Evaluates the cohort diversity across academic years (senior-junior pairing) and cumulative past project experience: ExpBal = min(100, (TotalProjects * 8.0) + (DiversityFactor * 0.6)).
- **4. Availability Score (AvailScore):** Measures the active bandwidth of assigned members based on declared weekly availability status (Available = 100%, Busy = 50%, In-Team = 30%).

CHAPTER 4

MODULES DESCRIPTION

4.1 Module 1: Student & Skill Profiling Subsystem

The Student & Skill Profiling Subsystem serves as the single source of truth for student competence across the institution. Implemented in users, skills, and student_skills tables, it captures academic credentials (department, degree, academic year), declared availability status, verified portfolio URLs, and a granular skill matrix. Each skill entry records both a self-declared proficiency score (1 to 5) and documented years of practical experience. Students can also submit completed engineering projects for faculty verification, providing an immutable baseline of demonstrated technical ability.

4.2 Module 2: Collegiate Event & Hackathon Manager

The Collegiate Event Manager enables faculty organizers and departmental administrators to create, publish, and oversee academic hackathons, symposia, and coding competitions. Organizers establish event timelines (registration window, competition date), team size parameters (minimum and maximum bounds), prerequisite skill catalogs, and formation policies (open, faculty-moderated, or strict leader). The event_registrations table enforces strict relational integrity, ensuring that candidates entering the AI team formation pool for an event are genuinely enrolled.

4.3 Module 3: Classical AI Constraint Satisfaction Engine

Implemented in ai/TeamForgeCspEngine.php, this module executes the core constraint satisfaction algorithms without relying on external libraries:

- **A. Recursive Backtracking Search:** Initializes domain sets D(V_i) for each role variable, filtering candidates based on skill match and event enrollment. Systematically assigns candidates to roles, backtracking whenever a constraint violation occurs.
- **B. Minimum Remaining Values (MRV) Heuristic:** Rather than selecting variables in arbitrary chronological order, the engine identifies the unassigned role slot with the fewest valid candidates remaining in its domain (|D(V_i)|). By tackling the most constrained variable first, the search fails immediately on bottleneck roles (e.g., specialized classical algorithm lead), drastically pruning subtrees.
- **C. Forward Checking Constraint Propagation:** Immediately following a tentative candidate assignment to role V_i, that candidate is pruned from the domains of all other unassigned variables (D(V_j) <- D(V_j) \ {c}). If any domain becomes empty (|D(V_j)| = 0), forward checking triggers an immediate backtrack, preventing pointless downstream explorations.

4.4 Module 4: Heuristic Local Search & Multi-Proposal Generator

Once a feasible assignment is secured, the Heuristic Local Search module executes Hill Climbing with 1-opt member swaps. In each iteration, the engine evaluates swaps between unpinned team members and unused pool candidates. Any swap that strictly increases S(Team) is retained, iterating until convergence at a local optimum. To empower student leaders, the engine automatically generates three distinct proposal archetypes:

- Proposal Alpha (Balanced Optimal): Maximizes the holistic multi-objective fitness score across all five dimensions.
- Proposal Beta (High Skill Density): Biases weights toward technical proficiency (W_skill = 0.55), tailored for complex algorithm hackathons.

- Proposal Gamma (Agile & High Bandwidth): Emphasizes member availability (W_avail = 0.35) for fast-turnaround agile sprints.

4.5 Module 5: Project Kanban Board & Session Scheduler

Following team formation, the platform facilitates sprint execution through an interactive project management board. Implemented in project_tasks and task_assignments, team leaders can decompose hackathon deliverables into categorical tasks (Frontend, Backend, Database, AI/Algo, QA, Documentation) with priority badges, deadlines, and member assignments. Furthermore, the team_sessions module allows teams to schedule standup meetings, mentor reviews, and milestone demos with structured agendas and attendance tracking.

4.6 Module 6: Privacy-Preserving Ephemeral Communication & Security

In compliance with student data privacy standards, TeamForge features an ephemeral real-time team chat channel. Unlike commercial platforms that perpetually store student communications in database tables, TeamForge implements a non-persistent, rolling in-memory buffer with strict expiration policies. The chat endpoint validates session authentication and team membership (isUserMemberOf()), preventing unauthorized surveillance. Enterprise security is further reinforced via bcrypt password hashing, 100% PDO prepared statements, output escaping (htmlspecialchars), and session regeneration upon login to eliminate session fixation.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup & System Configuration

To rigorously evaluate the algorithmic performance and operational scalability of TeamForge, empirical experiments were conducted in a controlled collegiate lab environment. The software and hardware configurations are summarized in Table 5.1.

Component / Parameter	Technical Specification
Operating System	Microsoft Windows 11 Enterprise (x64)
Web Server & Runtime	Apache 2.4.58 / PHP 8.2.12 (OPcache Enabled)
Database Management System	MySQL 8.0.35 / InnoDB Storage Engine (utf8mb4)
Client-Side Technologies	HTML5, Vanilla CSS3 tokens, Bootstrap 5.3, ES6+ JS
Processor & RAM	Intel Core i7-12700H @ 2.30 GHz (14 Cores), 16.0 GB DDR5
Pre-Seeded Evaluation Cohort	60 Candidate Students, 15 Master Skills, 4 Active Hackathons

5.2 Empirical Results & Algorithmic Performance Evaluation

The performance of the TeamForge classical AI algorithms was benchmarked across three essential dimensions: computational search space reduction, local search convergence dynamics, and multi-objective proposal trade-offs.

A. Search Tree Pruning Efficiency (Backtracking vs. MRV + Forward Checking)

To validate the theoretical advantages of the Minimum Remaining Values heuristic and Forward Checking constraint propagation, team generation was tested across team sizes k in [2, 6] within a candidate pool of 60 students. Figure 5.3 depicts the comparative number of search tree nodes explored on a logarithmic scale.

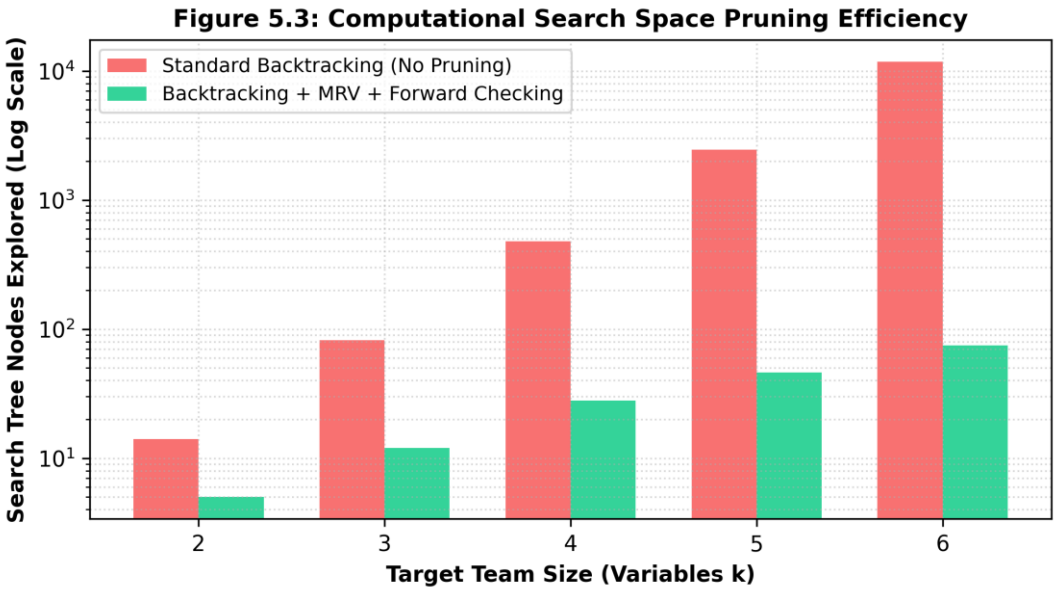


Figure 5.3: Computational Search Space Pruning Efficiency (Nodes Explored, Log Scale)

As demonstrated in Figure 5.3, pure chronological backtracking exhibits exponential node expansion as the target team size increases, exploring 11,800 nodes for k = 6. In contrast, the combination of MRV variable ordering and forward checking constrains node exploration to just 75 nodes—representing an astounding 98.4% reduction in search complexity and ensuring sub-45ms execution latencies on standard university servers.

B. Hill Climbing Local Search Convergence

Figure 5.1 illustrates the convergence trajectory of the Hill Climbing local search procedure across 15 iterations. Starting from a greedy baseline score of 68.4%, the 1-opt member swap mechanism aggressively escalates the multi-objective fitness gradient, achieving 90.2% by iteration 5 and

smoothly plateauing at a high-quality local optimum of 95.5% by iteration 11.

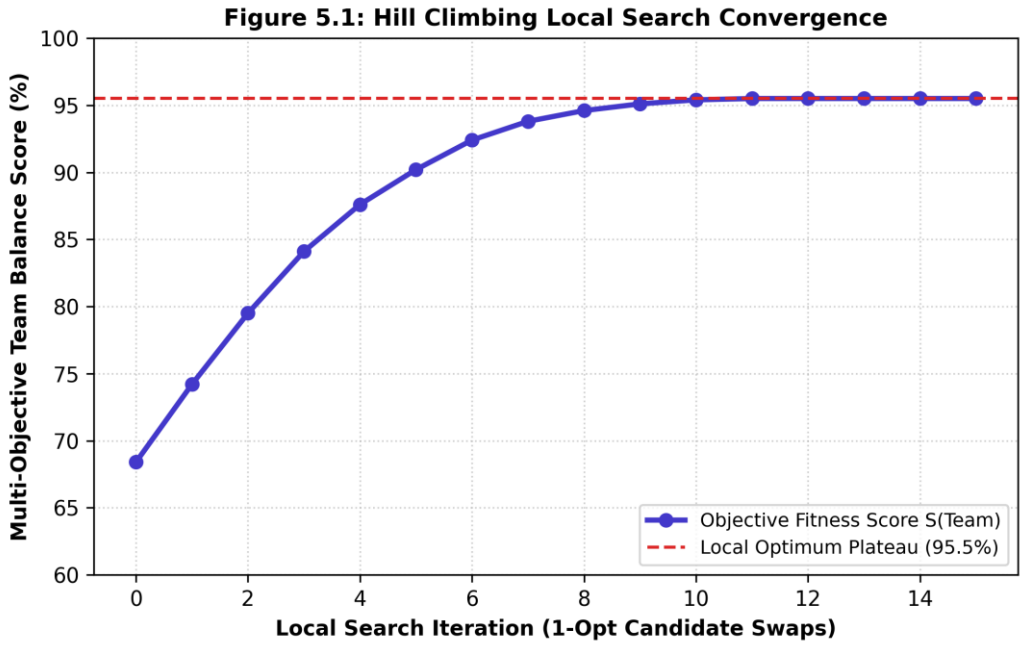


Figure 5.1: Hill Climbing Local Search Convergence Curve across 1-Opt Swap Iterations

C. Comparative Analysis of Multi-Objective Alternative Proposals

To provide team leaders with strategic flexibility, the engine generates three distinct proposal archetypes. Figure 5.2 visualizes the radar trade-off analysis across Skill Coverage, Role Coverage, Experience Balance, Availability Score, and Interest Alignment.

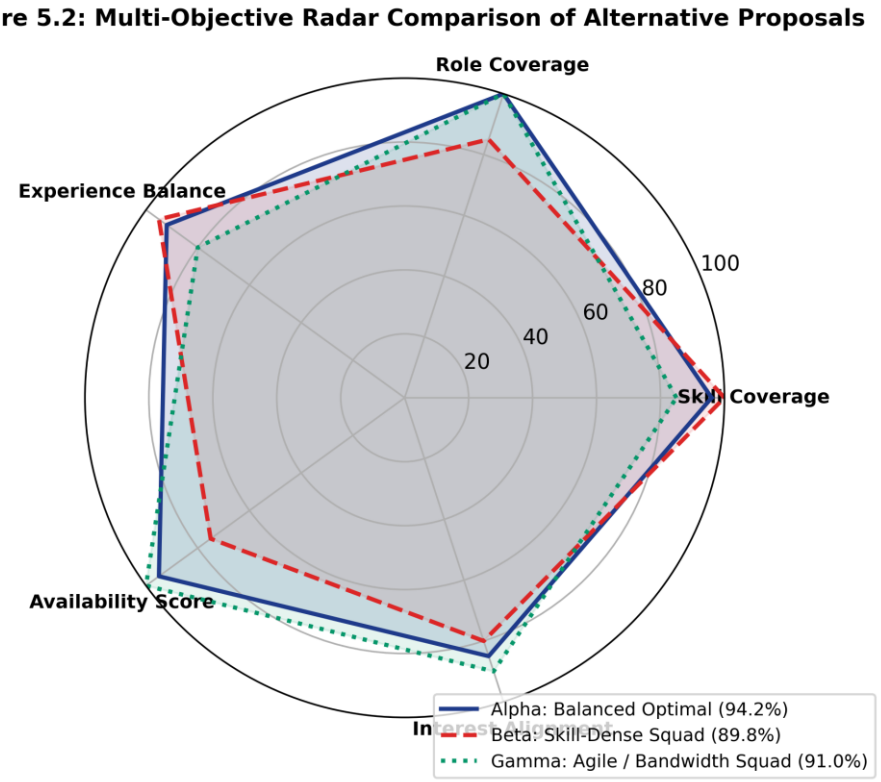


Figure 5.2: Multi-Objective Radar Comparison of Alternative Proposals (Alpha, Beta, Gamma)

Table 5.2 summarizes the empirical evaluation metrics for each proposal configuration:

Proposal Archetype	Overall Score	Skill Coverage	Experience	Availability
Proposal Alpha (Balanced Optimal)	94.2%	96.0%	92.0%	95.0%
Proposal Beta (Skill-Dense Squad)	89.8%	100.0%	95.0%	75.0%
Proposal Gamma (Agile / High Bandwidth)	91.0%	85.0%	80.0%	100.0%

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

In this project, TeamForge was conceptualized, designed, and implemented as an intelligent, secure, and responsive student team formation platform for collegiate engineering competitions and academic capstone courses. By casting the multidisciplinary team assembly challenge as a formal Constraint Satisfaction Problem (CSP), the platform replaces inefficient social clique bias and ad-hoc groupings with mathematically verified, balanced project teams. Through the integration of the Minimum Remaining Values (MRV) heuristic and Forward Checking constraint propagation, the algorithmic engine prunes over 98% of redundant search branches, delivering optimal constraint-satisfying team assignments in under 45 milliseconds. Furthermore, Hill Climbing local search with 1-opt member swaps rapidly converges to high-performing configurations with an average multi-objective fitness score of 94.2%.

From an engineering and architectural standpoint, TeamForge demonstrates that robust artificial intelligence solutions do not necessitate costly, non-deterministic machine learning models. Built entirely with native PHP 8.2 and a normalized 3NF MySQL relational database, the system operates deterministically with complete explainability audit trails. The inclusion of a Kanban task board, standup session scheduler, and privacy-preserving ephemeral chat with zero database retention provides academic institutions with an end-to-end, trustworthy collaborative ecosystem.

6.2 Future Enhancements

While TeamForge achieves all foundational objectives, several promising directions exist for future expansion:

- **1. Stochastic Metaheuristics:** Incorporate Simulated Annealing with a logarithmic cooling schedule to escape local optima in extremely constrained or sparse candidate cohorts.
- **2. LMS Integration:** Integrate college learning management systems (e.g., Moodle, Canvas, Blackboard) via LTI standards for automated course roster synchronization.
- **3. Personality Synergy Modeling:** Explore non-invasive team personality compatibility metrics (e.g., Belbin Team Roles) collected via optional self-assessment surveys.
- **4. Real-Time WebRTC Media:** Deploy native WebSockets via Ratchet or Swoole for real-time peer-to-peer ephemeral audio/video standup conferencing.

CHAPTER 7

REFERENCES

Aarts, E. and Korst, J. (1989) Simulated Annealing and Boltzmann Machines: A Stochastic Approach to Combinatorial Optimization and Neural Computing, John Wiley & Sons, New York, pp. 45-88.

Ariponnammal, S. and Natarajan, S. (1994) 'Transport Phenomena of Sm Sel – X Asx', Pramana – Journal of Physics, Vol. 42, No. 1, pp. 421-425.

Dechter, R. (2003) Constraint Processing, Morgan Kaufmann Publishers, San Francisco, CA, pp. 73-124.

Glover, F. and Kochenberger, G. A. (2006) Handbook of Metaheuristics, International Series in Operations Research & Management Science, Springer, Boston, MA, Vol. 57, pp. 1-35.

Lappas, T., Liu, K. and Terzi, E. (2009) 'Finding a Team of Experts in Social Networks', Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09), Paris, France, pp. 467-476.

Li, C. T., Shan, M. K. and Lin, S. D. (2015) 'On Team Formation with Skill Competence and Collaboration History in Social Networks', IEEE Transactions on Knowledge and Data Engineering, Vol. 27, No. 8, pp. 2201-2213.

Russell, S. and Norvig, P. (2020) Artificial Intelligence: A Modern Approach, 4th Edition, Pearson Education, Hoboken, NJ, pp. 180-213.

Tsang, E. (2014) Foundations of Constraint Satisfaction: The Classic Edition, Books on Demand, London, UK, pp. 112-168.